

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
Рязанский государственный радиотехнический университет
имени В.Ф. Уткина

Кафедра КТ

К защите
Руководитель работы:

дата, подпись

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ**

«Кроссплатформенная система управления и визуализации
мультимедийного контента для стриминговых сервисов»

Выполнил студент группы 2415
Тищенко А.С.

дата сдачи на
проверку, подпись

Руководитель работы
Филаткин С.В.

оценка

дата защиты,
подпись

Оглавление

Введение	4
2. Основания для разработки.....	6
2.1 Документы-основания.....	6
2.2 Цель разработки.....	6
3. Назначение разработки	8
3.1 Функциональное и эксплуатационное назначение программы	8
3.2 Краткое описание предоставляемых сервисов	9
4. Требования к программе	11
4.1 Требования к функциональным характеристикам.....	11
4.2 Требования к надёжности	12
4.3 Требования к производительности	13
4.4 Требования к составу и параметрам технических средств	14
4.5 Требования к информационной и программной совместимости.....	14
4.6 Требования к эргономике и технической эстетике.....	15
4.7 Требования к эксплуатации	15
4.8 Требования к защите информации	16
5. Требования к документированию	18
6. Техничко-экономические показатели.....	20
7. Стадии и этапы разработки	22
8. Порядок контроля и приёмки	25

8.1 Виды тестирования.....	25
8.2 Критерии приёмки.....	26
9. Приложения.....	27
Приложение А. Список используемых технологий.....	27
Приложение Б. Структура проекта.....	28
Приложение В. Примеры API запросов	28
Приложение Г. Задание на НИР.....	29

Введение

NowPlay — система управления и визуализации мультимедийного контента для стриминговых сервисов, представляющая собой комплексное программное решение для автоматического отображения информации о текущем воспроизводимом музыкальном треке непосредственно в прямых трансляциях и видеозаписях.

Программный продукт предназначен для стримеров на платформах Twitch и YouTube Gaming, контент-крейторов, создающих видеоконтент для различных платформ, пользователей программного обеспечения для стриминга (OBS Studio, Streamlabs OBS), музыкальных блогеров и обозревателей, создателей подкастов и аудиоконтента. Система применяется в прямых трансляциях, записи видеоконтента с отображением информации о воспроизводимой музыке, создании музыкальных шоу и подкастов, интеграции музыкального контента в игровые стримы.

Система осуществляет непрерывный мониторинг активных медиаплееров на компьютере пользователя через Windows Media Control API с частотой опроса один раз в секунду. При обнаружении воспроизведения музыки программа автоматически извлекает метаданные трека (исполнитель, название, альбом, длительность, текущая позиция воспроизведения) и инициирует загрузку обложки альбома из множественных источников с интеллектуальной приоритизацией. Поддерживаются все популярные медиаплееры: Spotify, Apple Music, YouTube Music, VLC Media Player, Windows Media Player, Groove Music, а также веб-браузеры при воспроизведении музыки на веб-сайтах.

Информация отображается через настраиваемые веб-виджеты, интегрируемые в OBS Studio через Browser Source. Предоставляется четыре профессиональных шаблона визуализации, оптимизированных для различных сценариев. Все шаблоны поддерживают автоматическое извлечение доминирующих цветов из обложки альбома для создания гармоничной цветовой палитры, Ambient Light эффект, плавные CSS анимации и прозрачный фон для наложения на любой контент.

Система построена на гибридной архитектуре: Backend реализован на Python 3.12+ с использованием асинхронного веб-сервера aiohttp 3.8+, библиотеки CustomTkinter 5.0+ для создания современного графического интерфейса. Компонент мониторинга медиаплееров разработан на C# .NET 6.0+ с использованием Windows Media Control API, что обеспечивает высокую производительность и нативную интеграцию с операционной системой Windows. Frontend построен на стандартных веб-технологиях HTML5/CSS3/JavaScript с применением Canvas API для обработки изображений и WebSocket API для real-time обновлений.

2. Основания для разработки

2.1 Документы-основания

Системный анализ системы выполняется на основании задания на научно-исследовательскую работу, предоставленного в приложении Г.

2.2 Цель разработки

Целью разработки является создание кроссплатформенной системы управления и визуализации мультимедийного контента для стриминговых сервисов с автоматическим отображением информации о текущем воспроизводимом треке, обеспечивающей высокую производительность, гибкую настройку визуализации и бесшовную интеграцию с популярными стриминговыми приложениями.

Анализ рынка программного обеспечения для отображения информации о музыке в стримах выявил ряд существенных проблем. Коммерческие решения (Streamlabs Ultra за \$19/месяц, XSplit Broadcaster Premium за \$25/месяц, Pretzel Rocks Pro за \$15/месяц) требуют значительных финансовых затрат, что создает барьер для начинающих стримеров и студентов. Бесплатные альтернативы обладают ограниченной функциональностью: Snip поддерживает только Spotify без визуализации обложек, OBS Now Playing имеет базовый функционал без настройки внешнего вида.

Серьезной проблемой является высокая нагрузка на системные ресурсы. Многие программы используют неоптимизированные алгоритмы обработки изображений, что приводит к нагрузке на CPU в диапазоне 15-35% при обработке обложек и избыточному использованию оперативной памяти (более 300 МБ). Это негативно влияет на производительность стриминга и игр. Большинство решений

работают только на Windows, ограничивая пользователей других операционных систем.

Индустрия стриминга демонстрирует динамичный рост. По данным Twitch Tracker (2024), на платформе Twitch зарегистрировано более 9.5 миллионов активных стримеров, YouTube Gaming насчитывает более 40 миллионов игровых каналов. Музыкальный контент играет ключевую роль: 78% стримеров используют фоновую музыку во время трансляций, 65% зрителей отмечают важность качественного аудиосопровождения. Интеграция музыки повышает вовлеченность аудитории на 23%.

Разработка NowPlay предоставит стримерам и контент-крейторам бесплатное, высокопроизводительное и функциональное решение с открытым исходным кодом, устраняя основные недостатки существующих аналогов.

3. Назначение разработки

3.1 Функциональное и эксплуатационное назначение программы

Основной функцией программы NowPlay является автоматическое отображение информации о текущем воспроизводимом мультимедийном контенте в стриминговых приложениях и видеозаписях. Система осуществляет непрерывный мониторинг активных медиаплееров на компьютере пользователя, извлекает метаданные о воспроизводимых треках и автоматически загружает обложки альбомов из множественных источников, предоставляя визуальное представление этой информации через настраиваемые веб-виджеты.

Программа решает задачу информирования зрителей о музыкальном контенте, звучащем во время трансляции или в записанном видео, без необходимости ручного ввода данных стримером. Это позволяет зрителям узнавать названия понравившихся треков, повышает профессионализм оформления контента и улучшает взаимодействие между стримером и аудиторией. Система работает полностью автоматически, не требуя вмешательства пользователя после первоначальной настройки.

Программа NowPlay предназначена для эксплуатации в трех основных режимах использования. Первый режим — интеграция с OBS Studio через Browser Source — является основным режимом работы для стримеров и контент-крейторов. Веб-виджет добавляется в сцену OBS как источник Browser Source, обеспечивая отображение информации о текущем треке непосредственно в трансляции.

Второй режим — standalone веб-интерфейс — предназначен для тестирования и предварительного просмотра виджетов перед

добавлением в OBS, использования на втором мониторе для личного контроля воспроизводимой музыки, а также для разработки и отладки новых шаблонов визуализации.

Третий режим — desktop-приложение с графическим интерфейсом — является центром управления всей системой, предоставляя интуитивный интерфейс для конфигурации, управления сервером и настройки параметров визуализации.

3.2 Краткое описание предоставляемых сервисов

Система NowPlay предоставляет комплекс взаимосвязанных сервисов для обеспечения полного цикла обработки мультимедийной информации — от мониторинга медиаплееров до визуализации данных в стриминговых приложениях.

Сервис мониторинга медиаплееров обеспечивает автоматическое обнаружение воспроизведения музыки через Windows Media Control API с частотой опроса один раз в секунду, извлечение базовых метаданных непосредственно от медиаплеера и автоматическую инициацию загрузки обложки при смене трека.

Сервис получения метаданных и обложек предоставляет загрузку обложек альбомов с интеллектуальной приоритизацией источников и fallback-механизмами. Система использует иерархию источников (Windows Media Thumbnail API, Last.fm, iTunes, Yandex Music, VK Music) для обеспечения максимальной доступности обложек.

Встроенный асинхронный веб-сервер предоставляет RESTful API и WebSocket для real-time обновлений. Сервер работает на localhost, обеспечивая безопасность, и предоставляет endpoints для получения информации о текущем треке, обложки, обновления конфигурации и установки WebSocket соединений.

Сервис визуализации предоставляет четыре профессиональных шаблона с настраиваемым дизайном, автоматическим извлечением цветов из обложки, Ambient Light эффектом и плавными анимациями. Все шаблоны оптимизированы для различных сценариев использования.

Графический интерфейс управления обеспечивает современный desktop-интерфейс для управления конфигурацией системы, запуска/остановки сервера, настройки параметров визуализации и мониторинга статуса работы всех компонентов.

4. Требования к программе

4.1 Требования к функциональным характеристикам

Программный продукт должен предоставлять полный набор возможностей для автоматического мониторинга, обработки и визуализации информации о воспроизводимом мультимедийном контенте.

Система должна обеспечивать автоматический мониторинг медиаплееров через Windows Media Control API с частотой опроса 1 секунда. Компонент должен автоматически определять активную медиасессию, извлекать метаданные трека (исполнитель, название, альбом, длительность, текущая позиция, статус воспроизведения) и инициировать загрузку обложки при смене трека. Поддерживаются медиаплееры: Spotify, Apple Music, YouTube Music, VLC, Windows Media Player, Groove Music, AIMP, foobar2000, MusicBee и веб-браузеры (Chrome, Firefox, Edge, Opera).

Система должна загружать обложки с приоритизацией источников: Windows Media Thumbnail API, Last.fm API, iTunes Search API, Yandex Music API, VK Music API. При недоступности источника автоматически переключается на следующий. Если все источники недоступны — используется дефолтная обложка. Требования: максимальное время загрузки 3 секунды, форматы PNG/JPEG/WebP, размер 600x600 пикселей (минимум 300x300), максимальный размер файла 5 МБ.

Встроенный асинхронный веб-сервер (aiohttp) работает на localhost:80. Предоставляет endpoints: POST /update_from_cs (обновления от C# компонента), GET /cover (получение обложки), GET /ws (WebSocket соединение), POST /update_config (обновление

конфигурации), GET /sources (список источников), GET /{template}.html (шаблоны визуализации). WebSocket обеспечивает real-time коммуникацию с троттлингом обновлений (минимум 100 мс между сообщениями). Время отклика API менее 10 мс, задержка WebSocket менее 100 мс.

Система должна предоставлять четыре профессиональных шаблона визуализации: вертикальный (300x400 пикселей), альтернативный (350x350 пикселей), минималистичный (250x80 пикселей), горизонтальный (600x120 пикселей). Все шаблоны должны поддерживать автоматическое извлечение доминирующих цветов из обложки, Ambient Light эффект (динамическая подсветка вокруг обложки), плавные CSS анимации и transitions, прозрачный фон для наложения на любой контент, адаптивное масштабирование текста при длинных названиях.

Desktop-приложение должно предоставлять современный интерфейс на базе CustomTkinter с поддержкой светлой и темной темы. Функционал: запуск/остановка сервера одной кнопкой, настройка порта и параметров визуализации, выбор активных источников обложек, отображение текущего трека и статуса системы, копирование URL виджетов для OBS, просмотр логов работы системы.

4.2 Требования к надёжности

Система должна обеспечивать стабильную работу в течение 24+ часов без перезапуска и деградации производительности. При сбое одного из компонентов (например, недоступность API источника обложек) система должна автоматически переключаться на альтернативные источники без прерывания работы. Все критические

ошибки должны логироваться с указанием времени, компонента и описания проблемы.

Система должна корректно обрабатывать ситуации отсутствия интернет-соединения (использование кэшированных обложек), недоступности медиаплееров (переход в режим ожидания), некорректных метаданных (отображение "Unknown Artist" / "Unknown Title"). При критическом сбое компонента система должна автоматически перезапускать его с сохранением состояния.

4.3 Требования к производительности

Система должна быть оптимизирована для минимальной нагрузки на системные ресурсы, чтобы не влиять на производительность стриминга и игр. Нагрузка на CPU должна составлять менее 2% при активном переключении треков и менее 0.5% в режиме ожидания. Использование оперативной памяти не должно превышать 200 МБ (Python компонент менее 150 МБ, C# компонент менее 50 МБ).

Время реакции на смену трека должно составлять менее 2 секунд от момента переключения в медиаплеере до обновления информации в веб-виджете. Загрузка обложки должна выполняться асинхронно и не блокировать обновление текстовой информации. Задержка WebSocket обновлений не должна превышать 100 мс.

Система должна эффективно использовать кэширование: обложки кэшируются на диске с автоматической очисткой старых файлов (максимум 500 МБ кэша), метаданные кэшируются в памяти для быстрого доступа, HTTP запросы к API используют connection pooling для снижения накладных расходов.

4.4 Требования к составу и параметрам технических средств

Минимальные системные требования: операционная система Windows 10 (64-bit) версии 1903 или выше, Windows 11; процессор с тактовой частотой не менее 1.8 ГГц (рекомендуется 2.4 ГГц); оперативная память не менее 4 ГБ (рекомендуется 8 ГБ); свободное место на диске не менее 500 МБ для установки и 1 ГБ для кэша обложек; интернет-соединение для загрузки обложек (минимум 1 Мбит/с).

Программные требования: Python 3.12 или выше, .NET 6.0 Runtime или выше, современный веб-браузер (Chrome 90+, Firefox 88+, Edge 90+) для просмотра виджетов, OBS Studio 27.0+ или Streamlabs OBS для интеграции виджетов в трансляцию.

Поддерживаемые медиаплееры: любые приложения, использующие Windows Media Control API (Spotify, Apple Music, YouTube Music, VLC, Windows Media Player, Groove Music, AIMP, foobar2000, MusicBee, веб-браузеры с воспроизведением медиа).

4.5 Требования к информационной и программной совместимости

Система должна обеспечивать совместимость с OBS Studio 27.0+ и Streamlabs OBS через стандартный механизм Browser Source. URL виджетов должны иметь формат `http://localhost:PORT/{template}.html`, где PORT — настраиваемый порт (по умолчанию 80), template — имя шаблона (vertical, alternative, minimal, horizontal).

API должно предоставлять данные в формате JSON с кодировкой UTF-8. Структура ответа `/update_from_cs` должна включать поля: artist, title, album, duration, position, status, cover_url. WebSocket сообщения должны использовать тот же формат JSON.

Система должна поддерживать интеграцию с внешними API: Last.fm API v2.0, iTunes Search API, Yandex Music API (неофициальное),

VK Music API. Все HTTP запросы должны использовать HTTPS для безопасности, таймаут запросов не более 5 секунд, автоматический retry при временных сбоях (максимум 3 попытки).

4.6 Требования к эргономике и технической эстетике

Графический интерфейс должен быть интуитивно понятным и не требовать специальных знаний для использования. Все элементы управления должны иметь понятные названия и подсказки. Интерфейс должен поддерживать светлую и темную темы с автоматическим переключением в зависимости от системных настроек.

Веб-виджеты должны иметь профессиональный дизайн с плавными анимациями и современной типографикой. Цветовая схема должна автоматически адаптироваться к обложке альбома для создания гармоничного визуального образа. Все шаблоны должны быть оптимизированы для различных разрешений экрана и корректно отображаться при масштабировании.

Текст в виджетах должен быть читаемым на любом фоне благодаря использованию теней и полупрозрачных подложек. При длинных названиях треков должно применяться адаптивное масштабирование шрифта или бегущая строка для обеспечения полной видимости информации.

4.7 Требования к эксплуатации

Система должна обеспечивать простоту установки и настройки для пользователей без технических знаний. Установка должна выполняться через единый инсталлятор, автоматически устанавливающий все необходимые зависимости (Python, .NET Runtime, библиотеки). Первый запуск должен включать мастер настройки с пошаговыми инструкциями.

Программа должна автоматически запускаться при старте системы (опционально, настраивается пользователем) и работать в фоновом режиме с минимальным интерфейсом в системном трее. Пользователь должен иметь возможность быстро открыть главное окно, остановить/запустить сервер и скопировать URL виджетов через контекстное меню трее.

Система должна предоставлять подробную документацию: руководство пользователя с иллюстрациями, инструкцию по интеграции с OBS Studio, описание всех шаблонов визуализации и их настроек, FAQ с решением типичных проблем, документацию API для разработчиков.

Обновления программы должны устанавливаться автоматически или по запросу пользователя с сохранением всех настроек и кэша. Система должна уведомлять о доступности новых версий через интерфейс приложения.

4.8 Требования к защите информации

Система должна обеспечивать безопасность данных пользователя и защиту от несанкционированного доступа. Веб-сервер должен работать только на localhost (127.0.0.1) и не принимать подключения из внешней сети. Порт сервера должен быть настраиваемым для избежания конфликтов с другими приложениями.

API ключи для внешних сервисов (Last.fm, Yandex Music, VK) должны храниться в зашифрованном виде в конфигурационном файле. Система не должна собирать и передавать личные данные пользователя на внешние серверы без явного согласия.

Все HTTP запросы к внешним API должны использовать HTTPS для защиты данных при передаче. Система должна валидировать все

входящие данные для предотвращения инъекций и других атак. Логи не должны содержать чувствительную информацию (API ключи, пароли).

Кэш обложек должен храниться в защищенной директории с ограниченным доступом. Система должна автоматически очищать временные файлы при завершении работы.

5. Требования к документированию

В комплект поставки программного изделия входит исчерпывающий набор документов, позволяющий пользователю полностью эксплуатировать и при необходимости развивать систему.

Обязательный перечень документации:

Техническое задание (настоящий документ), руководство пользователя с пошаговыми инструкциями и иллюстрациями, описание программы (техническая документация для разработчиков), инструкция по установке и настройке, документация API с описанием всех endpoints и форматов данных, описание архитектуры системы с диаграммами компонентов, руководство по интеграции с OBS Studio, FAQ с решением типичных проблем.

Документация должна быть предоставлена в двух форматах: PDF для финальной версии и Markdown для редактируемых исходников. Все документы должны содержать оглавление, нумерацию страниц и перекрестные ссылки. Иллюстрации должны быть высокого качества с подписями и пояснениями.

Руководство пользователя должно включать разделы: введение и обзор возможностей, системные требования и установка, первый запуск и настройка, интеграция с OBS Studio, описание шаблонов визуализации, настройка источников обложек, решение проблем и FAQ, контактная информация для поддержки.

Описание программы должно содержать: архитектуру системы с диаграммами, описание компонентов и их взаимодействия, спецификацию API с примерами запросов, описание алгоритмов обработки данных, требования к среде разработки, инструкции по

сборке из исходного кода, рекомендации по расширению функциональности.

6. Техничко-экономические показатели

Данный программный продукт обладает высокой технико-экономической и практической ценностью при разумной трудоёмкости реализации. С технической точки зрения проект опирается на проверенные и распространённые решения, что снижает риски разработки и упрощает последующее сопровождение.

Разработка NowPlay как бесплатного решения с открытым исходным кодом предоставляет значительную экономическую выгоду для целевой аудитории. Стримеры и контент-крийторы экономят от \$180 до \$300 в год на подписках коммерческих аналогов. Для начинающих стримеров и студентов это устраняет финансовый барьер входа в индустрию.

Снижение нагрузки на системные ресурсы (менее 2% CPU против 15-35% у аналогов) позволяет использовать систему на менее мощных компьютерах, что экономит средства на апгрейде оборудования. Оптимизация производительности также снижает энергопотребление при длительных трансляциях.

Использование современных технологий (Python 3.12+, .NET 6.0+, aiohttp, CustomTkinter) обеспечивает высокую производительность, масштабируемость и удобство разработки. Гибридная архитектура с разделением на Python и C# компоненты позволяет оптимально использовать преимущества каждой платформы.

Модульная архитектура системы упрощает добавление новых функций и интеграцию с дополнительными сервисами. Открытый исходный код позволяет сообществу вносить улучшения и адаптировать систему под специфические потребности.

Автоматизация процесса отображения информации о треках экономит время стримера и улучшает качество контента. Профессиональные шаблоны визуализации повышают эстетическую привлекательность трансляций. Поддержка множества медиаплееров и источников обложек обеспечивает универсальность применения.

Система способствует росту вовлеченности аудитории (по статистике на 23%), что напрямую влияет на монетизацию контента через подписки, донаты и рекламу. Для профессиональных стримеров это может означать увеличение дохода на 15-20%.

Ориентировочная трудоёмкость разработки составляет 400-500 человеко-часов, включая проектирование архитектуры (40 часов), разработку C# компонента мониторинга (80 часов), разработку Python backend (120 часов), создание веб-интерфейсов и шаблонов (100 часов), разработку desktop GUI (60 часов), тестирование и отладку (80 часов), документирование (40 часов).

При разработке в рамках ВКР срок реализации составляет 4-6 месяцев при работе 20-25 часов в неделю. Использование готовых библиотек и фреймворков значительно сокращает время разработки и повышает качество кода.

7. Стадии и этапы разработки

Разработка проекта делится на последовательные стадии, каждая из которых имеет свою цель, ожидаемые результаты и ориентировочные сроки выполнения.

Стадия 1. Проектирование и подготовка (2-3 недели).

Цель: создание детальной архитектуры системы и подготовка среды разработки. Результат: техническое задание, архитектурная документация с диаграммами компонентов, спецификация API и форматов данных, настроенная среда разработки с системой контроля версий, прототипы интерфейсов.

Стадия 2. Разработка C# компонента мониторинга (3-4 недели).

Цель: реализация компонента для мониторинга медиаплееров через Windows Media Control API. Результат: работающий C# сервис с интеграцией Windows Media Control API, извлечение метаданных треков, получение обложек через Windows Media Thumbnail API, HTTP клиент для отправки данных в Python backend, обработка ошибок и автоматический перезапуск при сбоях.

Стадия 3. Разработка Python backend (4-5 недель).

Цель: создание асинхронного веб-сервера и системы получения обложек. Результат: aiohttp сервер с RESTful API и WebSocket, интеграция с внешними API (Last.fm, iTunes, Yandex Music, VK), система кэширования обложек, обработка запросов от C# компонента, логирование и мониторинг работы системы.

Стадия 4. Разработка веб-интерфейсов и шаблонов (3-4 недели).

Цель: создание профессиональных шаблонов визуализации для OBS. Результат: четыре шаблона визуализации (вертикальный, альтернативный, минималистичный, горизонтальный), WebSocket клиент для real-time обновлений, алгоритм извлечения доминирующих цветов из обложки, Ambient Light эффект и плавные анимации, адаптивное масштабирование и обработка длинных названий.

Стадия 5. Разработка desktop GUI (2-3 недели).

Цель: создание графического интерфейса управления системой. Результат: CustomTkinter приложение с современным дизайном, управление запуском/остановкой сервера, настройка параметров и источников обложек, отображение текущего трека и статуса системы, копирование URL виджетов, просмотр логов.

Стадия 6. Тестирование и оптимизация (2-3 недели).

Цель: обеспечение стабильности и производительности системы. Результат: модульные тесты для критических компонентов, интеграционные тесты взаимодействия компонентов, нагрузочное тестирование и оптимизация производительности, тестирование на различных конфигурациях Windows, исправление выявленных ошибок и багов.

Стадия 7. Документирование и подготовка к релизу (1-2 недели).

Цель: создание полной документации и подготовка к публикации. Результат: руководство пользователя с иллюстрациями, техническая документация для разработчиков, инструкция по установке и интеграции с OBS, FAQ и решение типичных проблем, создание инсталлятора, публикация на GitHub с README и лицензией.

Общая длительность разработки: 17-24 недели (4-6 месяцев) при работе 20-25 часов в неделю.

8. Порядок контроля и приёмки

8.1 Виды тестирования

Приёмка программного изделия проводится на основе комплекса типов тестирования, покрывающего как функциональные, так и нефункциональные требования.

Функциональное тестирование включает модульные тесты критических компонентов (мониторинг медиаплееров, получение обложек, API endpoints), интеграционные тесты взаимодействия между C# и Python компонентами, тестирование WebSocket соединений и real-time обновлений, проверку корректности извлечения и обработки метаданных, тестирование fallback-механизмов при недоступности источников.

Тестирование производительности измеряет нагрузку на CPU и память при различных сценариях использования, время реакции на смену трека (целевое значение менее 2 секунд), задержку WebSocket обновлений (целевое значение менее 100 мс), время загрузки обложек из различных источников, стабильность работы при длительной эксплуатации (24+ часа).

Тестирование совместимости проверяет работу с различными медиаплеерами (Spotify, Apple Music, YouTube Music, VLC, браузеры), интеграцию с OBS Studio и Streamlabs OBS, корректность отображения в различных браузерах, работу на различных версиях Windows (10, 11).

Тестирование надёжности включает проверку обработки ошибок и исключений, автоматическое восстановление при сбоях компонентов, корректную работу при отсутствии интернет-соединения, устойчивость к некорректным данным от медиаплееров.

Тестирование безопасности проверяет ограничение доступа к веб-серверу только с localhost, защиту API ключей в конфигурации, отсутствие утечек чувствительных данных в логах, валидацию входящих данных.

8.2 Критерии приёмки

Приёмка считается успешной при выполнении следующих критериев:

1. Все функциональные требования реализованы и работают корректно согласно спецификации.
2. Система поддерживает все заявленные медиаплееры и источники обложек.
3. Производительность соответствует требованиям: CPU менее 2%, память менее 200 МБ, время реакции менее 2 секунд.
4. Все четыре шаблона визуализации корректно отображаются в OBS Studio.
5. Графический интерфейс интуитивно понятен и работает без ошибок.
6. Система стабильно работает 24+ часа без перезапуска.
7. Документация полная и соответствует требованиям раздела 5.
8. Отсутствуют критические ошибки и баги.
9. Код соответствует стандартам оформления и содержит комментарии.
10. Проект опубликован на GitHub с полной документацией и лицензией.

9. Приложения

Приложение А. Список используемых технологий

Backend:

- Python 3.12+
- aiohttp 3.8+ (асинхронный веб-сервер)
- aiofiles (асинхронная работа с файлами)
- Pillow (обработка изображений)
- requests (HTTP клиент)

Мониторинг медиаплееров:

- C# .NET 6.0+
- Windows.Media.Control API
- System.Net.Http (HTTP клиент)

Frontend:

- HTML5/CSS3/JavaScript
- Canvas API (обработка изображений)
- WebSocket API (real-time обновления)

Desktop GUI:

- CustomTkinter 5.0+
- tkinter (стандартная библиотека Python)

Внешние API:

- Last.fm API v2.0
- iTunes Search API
- Yandex Music API (неофициальное)
- VK Music API

Приложение Б. Структура проекта

```
NowPlay/
├── main.py                # Точка входа Python приложения
├── config.json            # Конфигурационный файл
├── requirements.txt       # Python зависимости
├── MediaMonitor/         # C# проект мониторинга
│   ├── MediaMonitor.csproj
│   ├── Program.cs
│   └── MediaMonitor.cs
├── ui/                   # Веб-интерфейсы
│   ├── vertical.html
│   ├── alternative.html
│   ├── minimal.html
│   └── horizontal.html
├── assets/               # Ресурсы (иконки, дефолтная обложка)
├── cache/                # Кэш обложек
├── logs/                 # Логи работы системы
└── docs/                 # Документация
    ├── USER_GUIDE.md
    ├── API_SPEC.md
    └── ARCHITECTURE.md
```

Приложение В. Примеры API запросов

Получение информации о текущем треке:

```
GET http://localhost:80/update_from_cs
Response: {
  "artist": "Artist Name",
  "title": "Track Title",
  "album": "Album Name",
  "duration": 240,
  "position": 120,
  "status": "playing",
  "cover_url": "/cover"
}
```

WebSocket подключение:

```
const ws = new WebSocket('ws://localhost:80/ws');
ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  // Обновление интерфейса
};
```

Приложение Г. Задание на НИР

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное
Учреждение высшего образования
«Рязанский государственный радиотехнический университет
им. В.Ф. Уткина»**

ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Кафедра космических технологий

«Утверждаю»
Зав. кафедрой КТ
_____ С.И. Гусев
«__» _____ 20__ г.

ЗАДАНИЕ НА ПРАКТИКУ НИР студента группы 2415

Тищенко Алексея Сергеевича

(Ф.И.О. студента)

Направление подготовки: 09.03.01 «Информатика и вычислительная техника»

Образовательная программа: «Системный анализ и инжиниринг информационных процессов»

Трудоемкость НИР: 2 зачетных единицы, 72 часа.

Сроки проведения практики: с «1» сентября 2025 г. по «13» декабря 2025 г.

Место выполнения практики: 390005, г. Рязань, ул. Гагарина, 59/1

Руководитель: Филаткин Сергей Владимирович

Рабочий график (план) работы студента

№	Наименования задачи мероприятия	Сроки выполнения	Форма окончания
1	Постановка задачи	01.09.2025 – 08.09.2025	Утвержденная формулировка целей и задач
2	Сбор исходных данных	09.09.2025 – 18.09.2025	Набор исходных данных и нормативных документов (аналоги, ГОСТы, стандарты источники данных)
3	Выбор и обоснование критериев эффективности и качества разрабатываемой программы	19.09.2025 – 26.09.2025	Перечень критериев с обоснованием
4	Предварительный выбор методов решения задач	27.09.2025 – 06.10.2025	Анализ способ решений задач на основе различных методов и технологий
5	Обоснование целесообразности применения ранее разработанных программ	07.10.2025 – 14.10.2025	Анализ существующих программ и их применения для решения поставленных задач
6	Определение требований к техническим средствам	15.10.2025 – 22.10.2025	Спецификация технических требований
7	Обоснование принципиальной возможности решения поставленной задачи	23.10.2025 – 30.10.2025	Анализ и обоснование возможности реализации способов решения поставленных задач
8	Определений требований к программе	31.10.2025 – 10.11.2025	Перечень функциональных и нефункциональных требований
9	Определение стадий, этапов и сроков разработки программы и документации	11.11.2025 – 18.11.2025	Календарный план-график работ
10	Выбор языков программирования	19.11.2025 – 25.11.2025	Обоснование выбора перечня инструментов разработки
11	Согласование и утверждение научно-исследовательской работы	26.11.2025 – 13.12.2025	Утвержденный отчет по научно-исследовательской работе

Дата выдачи задания: «__» _____ 20__ г.

Руководитель практики

_____/_____/_____
(подпись / расшифровка подписи)

Задание принял
к исполнению
(обучающийся)

_____/_____/_____
(подпись / расшифровка подписи)